

EJERCICIO 1 (MARCOS, 1 punto)

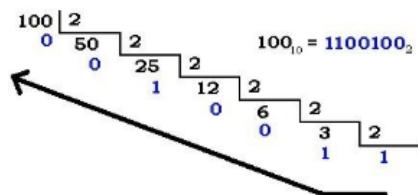
Desarrollar el método de extracción de un **nodo cualquiera** de una lista `dll_t<T>`, con la siguiente cabecera:

```
C/C++
void dll_t<T>::extract(dll_node_t<T> *nodo);
```

teniendo en cuenta también los casos extremos de extracción por el frente y el final. Hacer uso de los métodos ya desarrollados dentro de la clase `dll_t<T>`.

EJERCICIO 2 (MARCOS, 2 puntos)

El clásico algoritmo de conversión de un número n en base decimal (base 10) a otra base b , consiste en dividir n entre la base b sucesivamente mientras el dividendo sea mayor o igual que b , a continuación, se coge el último cociente hasta el primer resto en orden inverso. Por ejemplo, si se convierte el número 100 en base 10 a base 2, las operaciones a realizar serían las siguientes:



Dado un número entero positivo n y una base entera b , se pide desarrollar un **algoritmo iterativo** de conversión de dicho número entero a cualquier base b (con $1 < b < 10$), devolviendo un `vector_t<unsigned>` con el número en dicha base, y cuya cabecera debe ser:

```
C/C++
vector_t<unsigned> to_base(const unsigned n, const unsigned short b);
```

Usando el ejemplo anterior, el resultado esperado por pantalla en caso de ser base 8 sería `[1,4,4]`.

NOTA: se valorará la **eficiencia** en el desarrollo del algoritmo y uso de las estructuras de datos adecuadas.

EJERCICIO 3 (MARCOS, 2 puntos)

El algoritmo de *shunting yard* convierte una expresión en notación **infija** a otra en **postfija**, para que pueda ser evaluada por un ordenador (usando RPN por ejemplo). El algoritmo se define así:

1. Crear una **estructura de datos** vacía donde se almacenarán solo los símbolos +, -, *, y /.
2. Crear una **estructura de datos** vacía de salida donde se formará la expresión postfija.
3. Recorrer la cadena de izquierda a derecha, carácter a carácter:
 - Si el carácter es un dígito ('0'..'9'): insertarlo en la salida.
 - En otro caso (operador '+', '-', '*', '/'):
 - Mientras la estructura de datos no esté vacía y la precedencia del operador en el tope sea mayor o igual que la del operador actual, quitar el operador del tope e insertarlo en la salida.
 - Insertar el operador actual en la estructura de datos
4. Quitar todos los operadores que queden en la estructura de datos e insertarlos en la salida.

La cabecera de la función será la siguiente:

```
C/C++
queue_l_t<char> infixToPostfix(const vector_t<char>& expr);
```

Pueden hacer uso de las siguientes funciones:

```
C/C++
bool isdigit(char c); // devuelve true si 'c' es un dígito

inline int precedencia(char op) // indica la precedencia de un operador
{ if (op == '+' || op == '-') return 1;
  if (op == '*' || op == '/') return 2;
  return 0;
}
```

Por ejemplo, si la expresión de entrada es $7 + 3 * 2 - 5 / 1$, la salida esperada será:

$7\ 3\ 2\ *\ +\ 5\ 1\ /\ -$

Nota: en la corrección de este ejercicio se tendrá muy en cuenta la eficiencia en el desarrollo de la solución propuesta y la elección de las **estructuras de datos**.

EJERCICIO 4 (RIERA, 2 puntos)

Se desea implementar un algoritmo que permita encontrar **todas las posibles particiones de un conjunto de números enteros positivos** en dos subconjuntos cuya **suma total sea igual**. Para ello, se utilizará un enfoque recursivo basado en *backtracking*. El conjunto de entrada está representado por un array de enteros positivos `values[]` y se deben explorar todas las combinaciones posibles para decidir, mediante un vector binario `x[]`, en qué subconjunto se ubica cada elemento:

- `x[i] = 1` si el elemento `values[i]` se asigna al subconjunto A.
- `x[i] = 0` si se asigna al subconjunto B.

Se proporciona el código principal del programa, incluyendo la función `main` y una función auxiliar `write()` que imprime una partición válida. **Completar la función recursiva `generate`** que explore todas las asignaciones posibles de elementos a subconjuntos, y muestre únicamente aquellas que generen una partición equitativa.

```

C/C++
void write(const int x[], const int values[], const int n)
{ cout << "Conjunto A: ";
  for (int j = 0; j < n; ++j)
    if (x[j]) cout << values[j] << " ";
  cout << endl;
  cout << "Conjunto B: ";
  for (int j = 0; j < n; ++j)
    if (!x[j]) cout << values[j] << " ";
  cout << endl;
  cout << endl;
}

void generate(int x[], int i, const int sumA, const int n, const int totalSum,
             const int values[])
{
  // ...
}

int main()
{ const int n = 8;
  const int values[n] = {3, 1, 5, 9, 12, 6, 2, 4};
  int x[n];
  int totalSum = accumulate(values, values + n, 0);
  cout << "Suma total de los valores: " << totalSum << endl;
  generate(x, 0, 0, n, totalSum, values);
  return 0;
}

```

EJERCICIO 5 (RIERA, 2 puntos)

Se dice que una matriz cuadrada es **simétrica respecto a su diagonal principal** si cada elemento por encima de la diagonal coincide con su simétrico por debajo de ella. Es decir, una matriz A de tamaño $m \times m$ es simétrica si se cumple: $A(i, j) = A(j, i)$, $1 \leq i < j \leq m$. Por ejemplo, la siguiente matriz sí es simétrica:

```

C/C++
1 2 3
2 4 5
3 5 6

```

Se proporciona la clase `matrix_t<T>` con los siguientes métodos:

- `int get_m() const`: devuelve el número de filas/columnas.
- `T& operator()(int i, int j)`: devuelve una referencia al elemento en la fila i y columna j , indexado desde 1.

Se pide:

- a) Implementa una **función iterativa eficiente** que determine si la matriz cuadrada **A** es simétrica respecto a su diagonal principal. La función debe devolver **true** si es simétrica y **false** en caso contrario. (0,5 puntos)

```
C/C++
bool es_simetrica(const matrix_t<int> &A);
```

- b) Implementa una función recursiva: (1,5 puntos)

```
C/C++
bool es_simetrica_rec(const matrix_t<int> &A, int i, int j);
```

que sea llamada como:

```
C/C++
bool es_simetrica(const matrix_t<int> &A)
{ return es_simetrica_rec(A, 1, 2);
}
```

EJERCICIO 6 (RIERA, 1 punto)

Una matriz cuadrada de tamaño $n \times n$ se puede trasponer **sobre sí misma** si se intercambian simétricamente los elementos respecto a la diagonal principal. Es decir, por cada par de índices (i, j) con $i < j$, se debe realizar el intercambio $A(i, j) \leftrightarrow A(j, i)$. A fin de minimizar el número de operaciones, sólo es necesario recorrer una de las dos submatrices triangulares **excluyendo la diagonal**. En total, deben realizarse $\frac{n(n-1)}{2}$ intercambios. Implementa una función:

```
C/C++
void trasponer(matrix_t<int> &A);
```

que modifique **A** directamente, trasponiendo sus elementos sobre sí misma de forma eficiente, **sin usar** matrices auxiliares, y recorriendo únicamente la submatriz triangular superior sin incluir la diagonal.

C/C++

```

class vector_t {
public:
    vector_t(const int = 0);
    ~vector_t();
    void resize(const int);
    int get_size(void) const;
    T& at(const int);
    T& operator[](const int);
    const T& at(const int) const;
    const T& operator[](const int) const;
    void write(ostream& = cout) const;
    void read(istream& = cin);
private:
    T *v_;
    int sz_;
    void build(void);
    void destroy(void);
};

```

C/C++

```

class matrix_t {
public:
    matrix_t(const int = 0, const int = 0);
    ~matrix_t();
    void resize(const int, const int);
    int get_m(void) const;
    int get_n(void) const;
    T& at(const int, const int);
    T& operator()(const int, const int);
    const T& at(const int, const int) const;
    const T& operator()(const int, const int) const;
    void write(ostream& = cout) const;
    void read(istream& = cin);
private:
    int m_, n_;
    vector_t<T> v_;
    int pos(const int, const int) const;
};

```

C/C++

```

class dll_t {
public:
    dll_t(void);
    ~dll_t(void);
    dll_node_t<T>* get_tail(void) const;
    dll_node_t<T>* get_head(void) const;
    int get_size(void) const;
    bool empty(void) const;
    void push_back(dll_node_t<T>*);
    void push_front(dll_node_t<T>*);
    dll_node_t<T>* pop_back(void);
    dll_node_t<T>* pop_front(void);
    dll_node_t<T>* erase(dll_node_t<T>*);
    ostream& write(ostream& = cout) const;
private:
    dll_node_t<T> *head_, *tail_;
    int sz_;
};

```

C/C++

```

class stack_l_t {
public:
    stack_l_t(void);
    ~stack_l_t(void);
    void push(const T&);
    void pop(void);
    const T& top(void) const;
    bool empty(void) const;
    ostream& write(ostream& os = cout) const;
private:
    dll_t<T> l_;
};

```

C/C++

```

class queue_l_t {
public:
    queue_l_t(void);
    ~queue_l_t(void);
    bool empty(void) const;
    int size(void) const;
    void push(const T&);
    void pop(void);
    const T& front(void) const;
    const T& back(void) const;
    ostream& write(ostream& os = cout) const;
private:
    dll_t<T> l_;
};

```

EJERCICIO 1

```

C/C++
void dll_t<T>::extract(dll_node_t<T> *nodo)
{ if (nodo == get_head())      nodo = pop_front();
  else if (nodo == get_tail()) nodo = pop_back();
  else
  { dll_node_t<T> *prev = nodo->get_prev();
    dll_node_t<T> *next = nodo->get_next();
    prev->set_next(next);
    next->set_prev(prev);
    nodo->set_next(NULL);
    nodo->set_prev(NULL);
    sz--;
  }
}

```

EJERCICIO 2

```

C/C++
vector_t<unsigned> to_base(const unsigned n, const unsigned short b)
{ assert(n > 0);
  unsigned n_ = n;
  stack_l_t<unsigned> pila;

  int i = 0;
  while (n_ >= b)
  { pila.push(n_ % b);
    n_ = n_ / b;
    i++;
  }
  pila.push(n_);
  i++;

  vector_t<unsigned> v(i);
  i = 0;
  while (!pila.empty()) {
    v[i++] = pila.top();
    pila.pop();
  }

  return v;
}

```

EJERCICIO 3

```

C/C++
queue_l_t<char> infixToPostfix(const vector_t<char>& expr)
{ stack_l_t<char> pila;
  queue_l_t<char> salida;

  for (int i = 0; i < expr.get_size(); ++i)
  { char c = expr[i];
    if (isdigit(c)) salida.push(c);
    else
    { while (!pila.empty() && precedencia(pila.top()) >= precedencia(c))
      { salida.push(pila.top());
        pila.pop();
      }
      pila.push(c);
    }
  }
  while (!pila.empty())
  { salida.push(pila.top());
    pila.pop();
  }
  return salida;
}

```

EJERCICIO 4

```

C/C++
void generate(int x[], int i, const int sumA, const int n, const int totalSum,
             const int values[])
{
    if (i == n)
    {
        int sumB = totalSum - sumA;
        if (sumA == sumB) write(x, values, n);
    }
    else
    {
        x[i] = 0;
        generate(x, i + 1, sumA, n, totalSum, values);
        x[i] = 1;
        generate(x, i + 1, sumA + values[i], n, totalSum, values);
    }
}

```

EJERCICIO 5

```

C/C++
es_simetrica(const matrix_t<int> &A)
{
    const int m = A.get_m();
    bool simetrica = true;
    for (int i = 1; i <= m; ++i)
        for (int j = i + 1; j <= m; ++j)
            if (A(i,j) != A(j,i))
                simetrica = false;

    return simetrica;
}

bool es_simetrica_rec(const matrix_t<int> &A, int i, int j)
{
    const int m = A.get_m();
    if (i > m) return true;
    if (j > m) return es_simetrica_rec(A, i + 1, i + 2);
    if (A(i, j) != A(j, i)) return false;
    return es_simetrica_rec(A, i, j + 1);
}

```

EJERCICIO 6

```

C/C++
void trasponer(matrix_t<int> &A)
{
    const int n = A.get_m();
    for (int i = 1; i <= n; ++i)
        for (int j = i + 1; j <= n; ++j)
            std::swap(A(i, j), A(j, i));
}

```

Última modificación: martes, 10 de marzo de 2026, 11:06

Universidad de La Laguna

Pabellón de Gobierno, C/ Padre Herrera s/n. | 38200 | Apartado Postal 456 | San Cristóbal de La Laguna | España | (+34) 922 31 90 00

